

ANASIEZE IKENNA – CLOUD & AI ENGINEER

Project: Azure-Powered Multimodal AI Applications

Overview

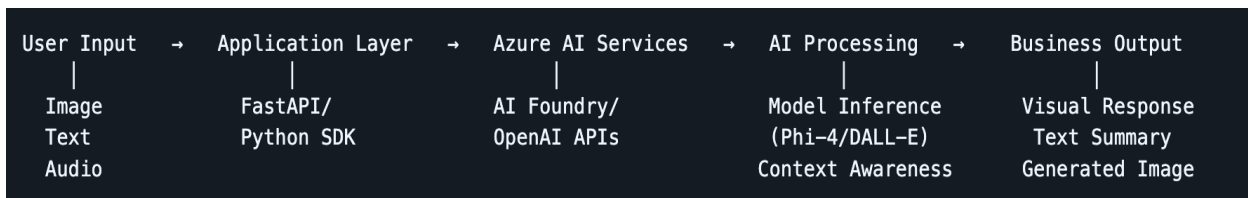
I developed an enterprise-grade suite of multimodal generative AI applications using Microsoft Azure's cloud platform. This project demonstrates end-to-end expertise in building production-ready AI solutions that process and generate multiple data types (text, images, and audio) to solve real-world business problems.

What I Built

Three distinct but interconnected AI applications:

- ⇒ **Vision-Enabled Chat Assistant** - A multimodal AI system that understands both images and text, allowing users to upload photos with questions and receive context-aware responses using Azure AI Foundry and the Phi-4 model.
- ⇒ **AI Image Generator** - A text-to-image creation tool that generates high-quality visuals from text descriptions using **Azure OpenAI's DALL-E 3 model**, enabling automated content creation for various business needs.
- ⇒ **Audio Processing Assistant** - An AI system that summarizes voice messages and extracts urgent requests using Azure AI Foundry with Phi-4, automating customer service workflows in retail and supply chain operations.

Architecture



Tech Stack

Component	Tools Used
Programming	Python 3.10
AI Services	Azure AI Foundry, Azure OpenAI
Frameworks	FastAPI, Azure OpenAI SDK
AI Models	Phi-4 Multimodal, DALL-E 3
Security	Azure Identity, DefaultAzureCredential
Development Tools	Azure AI Playground, Cloud Shell, GitHub

⇒ Vision-Enabled Chat Assistant:

○ Flow:

1. User uploads an image + text prompt
2. Fast API backend receives request
3. Request sent to Azure AI Foundry endpoints
4. Phi-4 multimodal model processes text + image
5. AI response (blending vision + language) is returned to the user

○ Methodology

1. Designed and implemented the **chat application backend** using Python and Fast API
2. Integrated **Azure AI multimodal inference** APIs for image + text understanding
3. Tested and optimized the app in **Azure AI Playground** before full deployment
4. Managed cloud resource setup and teardown for cost efficiency and scalability

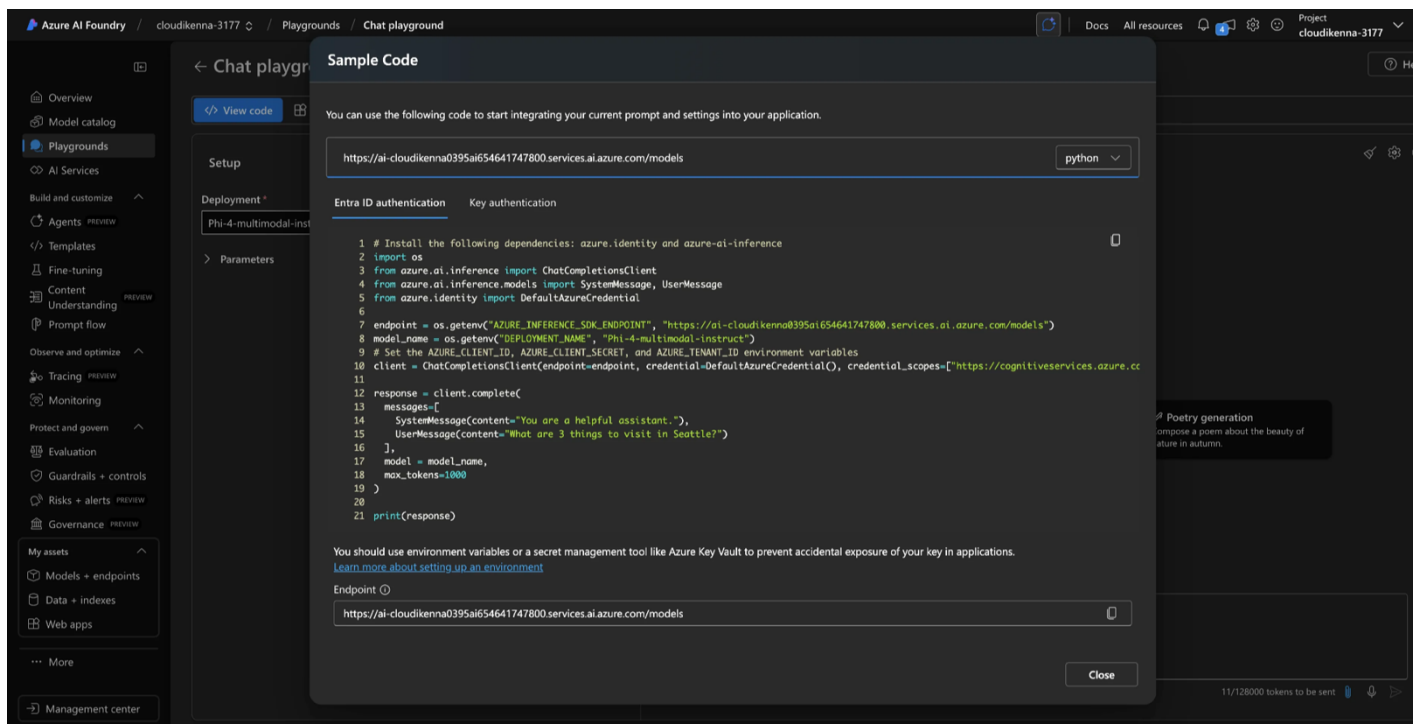
- **Set Up**

- Clone the repo and install requirements

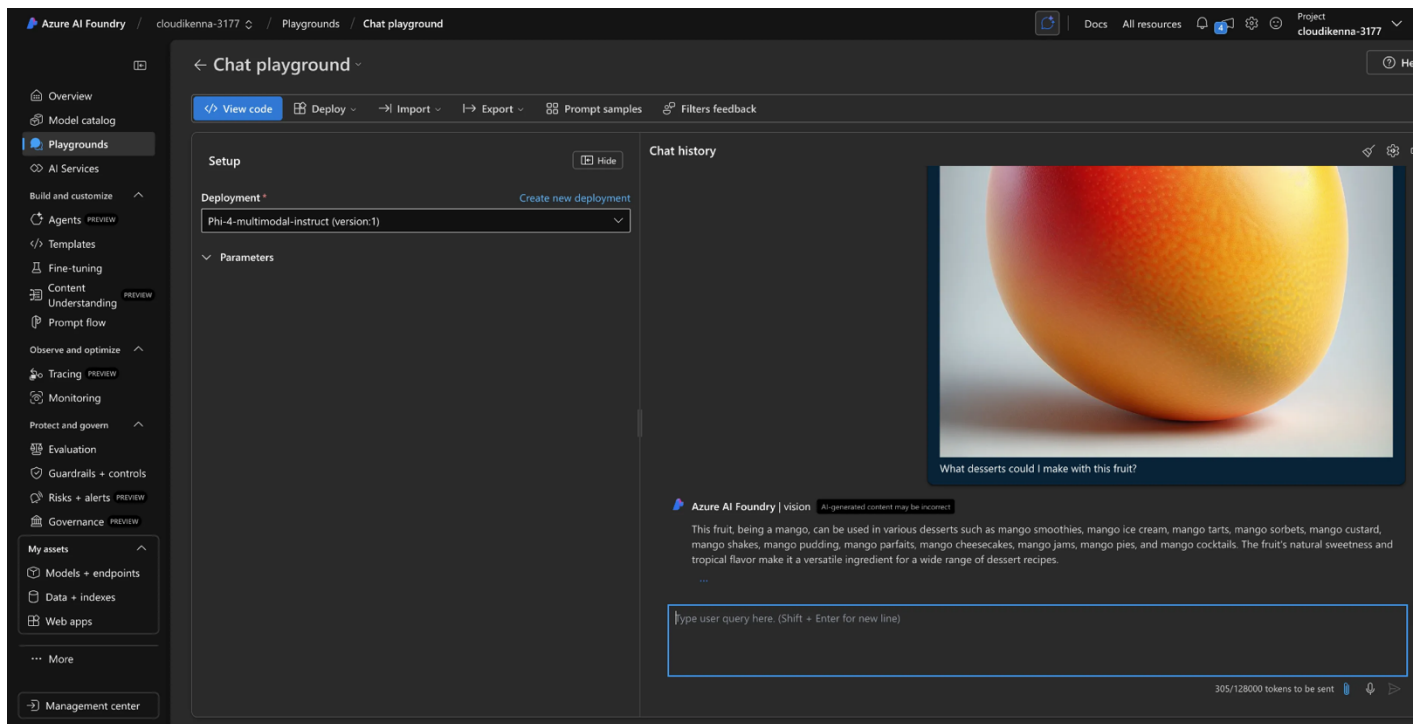
- `cd chat-assistant`
 - `pip install -r requirements.txt`

- Configure `chat-app.py`

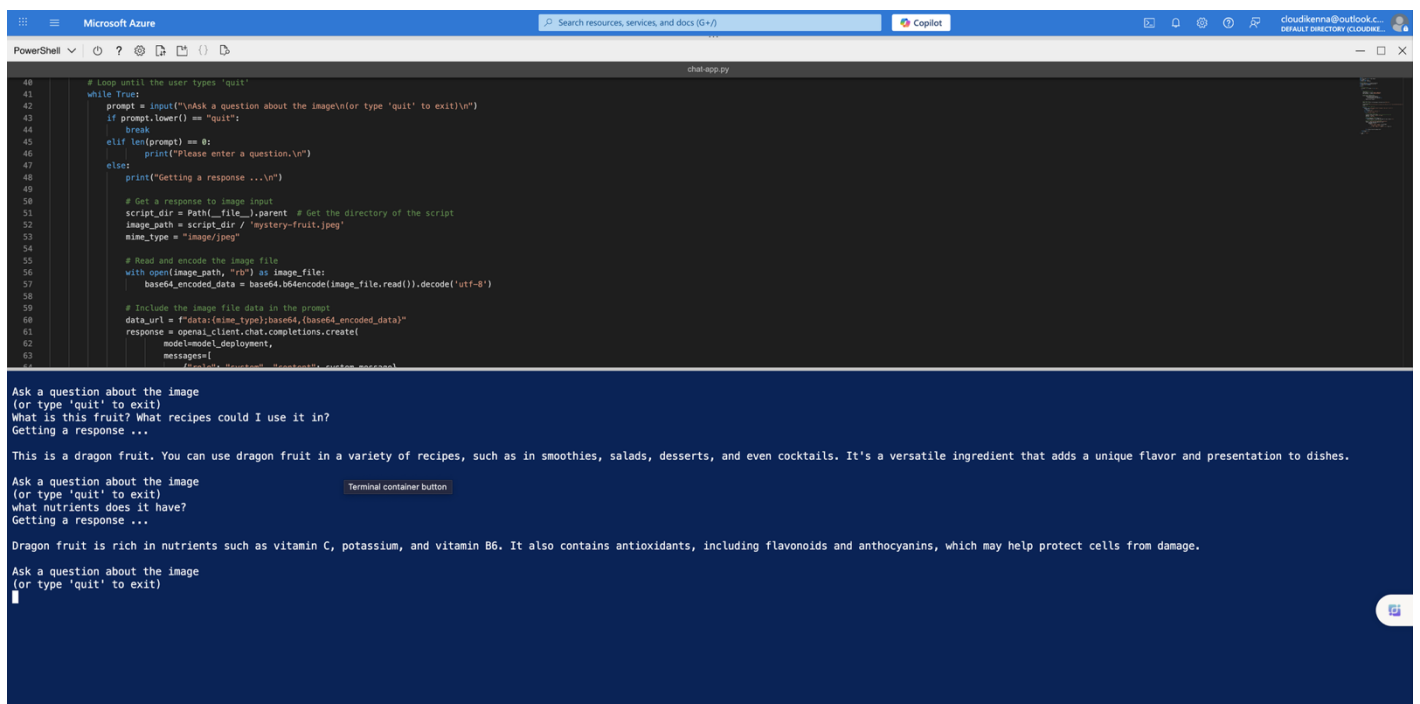
- Selecting and Creating Model



- Testing Model in Playground



- Creating a client application



⇒ **AI Image Generator:**

- **Flow:**

1. User enters a text prompt (e.g., “a robot eating spaghetti in a restaurant”)
2. Request is sent to the Azure DALL·E model endpoint
3. Model generates an image and returns a URL
4. The client app downloads and saves the image locally

- **Methodology**

1. Deployed DALL·E 3 in Azure AI Foundry and tested with iterative prompts.
2. Built Python client using Azure OpenAI SDK with secure authentication.
3. Implemented prompt-to-image pipeline with automatic PNG download.
4. Optimized resource management for cost efficiency and scalability.

- **Set Up**

- Clone the repo and install requirements

- *cd chat-assistant*
 - *pip install -r requirements.txt*

- Configure **image-app.py**

- Selecting and Creating Model

The screenshot shows the 'dall-e-3' deployment page in the Azure AI Foundry. The left sidebar contains navigation links for Overview, Model catalog, Playgrounds, Build and customize, Agents, Templates, Fine-tuning, Content Understanding, Observe and optimize, Tracing, Monitoring, Protect and govern, Evaluation, Guardrails + controls, Risks + alerts, Governance, Azure OpenAI, Assistant vector stores, Data files, My assets, Models + endpoints, and Management center. The main content area is divided into several sections: 'Details' with 'Request quota', 'Edit', and 'Delete' buttons; 'Endpoint' showing the Target URI and Authentication type (Key); 'Deployment info' with a table of deployment details; 'Capacity units' showing 1 unit and rate limits; and 'Monitoring & safety' with a content filter. On the right, there are dropdowns for Language (REST), SDK (curl), and Authentication type (Key Authentication). Below these is the 'Get Started' section with two sub-sections: '1. Authentication using API Key' and '2. Run a basic code sample'. The first section provides instructions for using bash, powershell, and Windows command prompt to set the API key. The second section shows a curl command to generate an image.

Deployment info	
Name	dall-e-3
Provisioning state	Succeeded
Deployment type	Standard
Created by	cloudikenna@outlook.com
Modified by	cloudikenna@outlook.com
Created on	2025-08-31T21:53:18.0632783Z
Modified on	Aug 31, 2025 7:23 PM
Version upgrade policy	Once a new default version is available

Capacity units	
1	
Rate limit (Tokens per minute)	3
Rate limit (Requests per minute)	3

```
curl -X POST "https://cloud-mf0866ff-swedencentral.cognitiveservices.azure.com/openai/deployments/dall-e-3/images/generate" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <AZURE_API_KEY>" \
-d '{
  "model": "dall-e-3",
  "prompt": "A baby in a Gucci suit sitting on a chair",
  "n": 1,
  "style": "vivid",
  "quality": "standard"
}'
```

- Testing Model in Playground

The screenshot shows the 'Images playground' interface in the Azure AI Foundry. The left sidebar is the same as the previous screenshot. The main content area shows the 'Images playground' with a 'Deployments' dropdown set to 'dall-e-3'. Below this are three generated images of babies in expensive suits. At the bottom, there is a text input field with the prompt 'Generate images of a babies in an expensive suit', a '1024x1024' resolution selector, a 'Style' dropdown, a 'Quality' dropdown, and a 'Generate' button. A message at the bottom states: 'Messages in the image playground are visible to anyone with access to this resource and using the API.'

Generate images of a babies in an expensive suit

1024x1024 Style Quality Generate

Messages in the image playground are visible to anyone with access to this resource and using the API.

⇒ **Audio Processing Assistant:**

○ **Flow:**

1. Customer leaves an audio message (e.g., *"We need 100 crates of avocados for urgent delivery"*)
2. Audio file is encoded into base64 and sent with a text prompt to the **Phi-4 multimodal model**
3. Model processes both the audio and text input
4. Response is returned as a **summarized insight** (e.g., *"Urgent avocado order, time-sensitive"*)

○ **Methodology**

1. Deployed the Phi-4 multimodal instruct model in Azure AI Foundry.
2. Tested audio summarization in the Azure AI Playground with sample voice messages.
3. Developed a Python client app with Azure OpenAI SDK and Azure Identity for secure access.
4. Encoded MP3 files into base64 format and passed them to the model with text prompts.
5. Implemented logic to provide concise summaries and mark time-sensitive requests.
6. Ensured cost control and scalability with resource management and cleanup

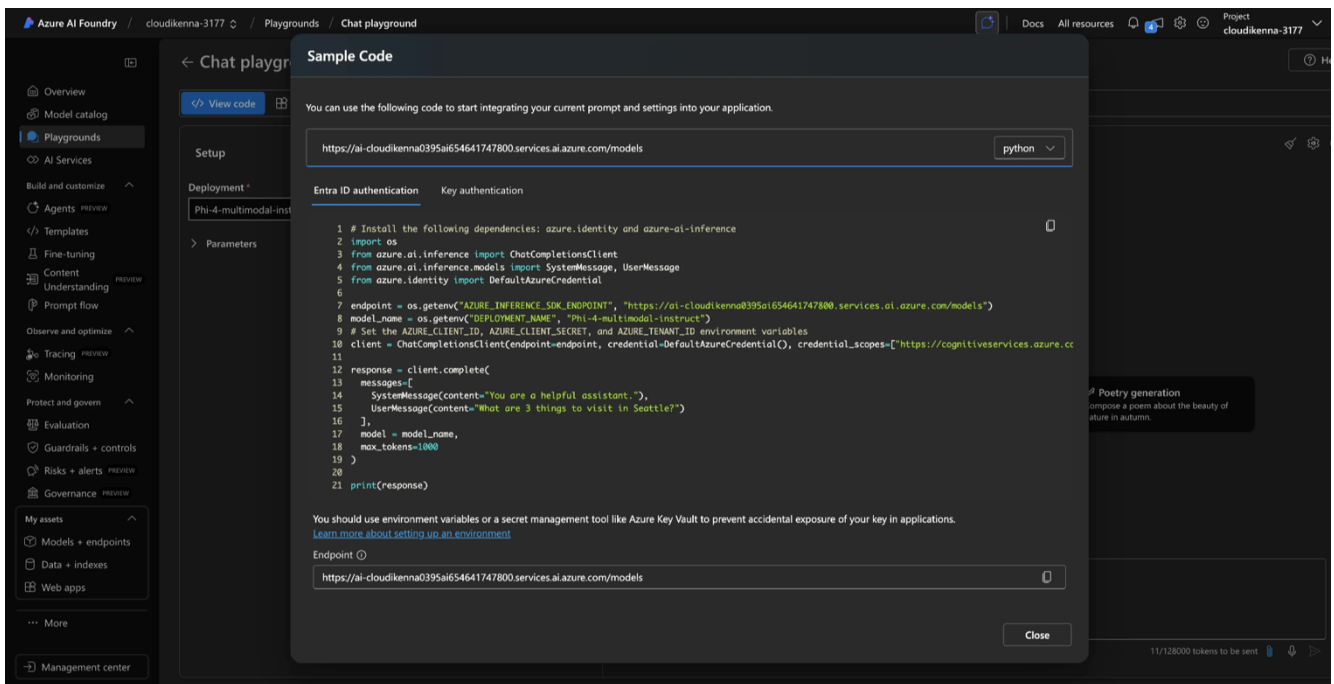
• **Set Up**

- Clone the repo and install requirements

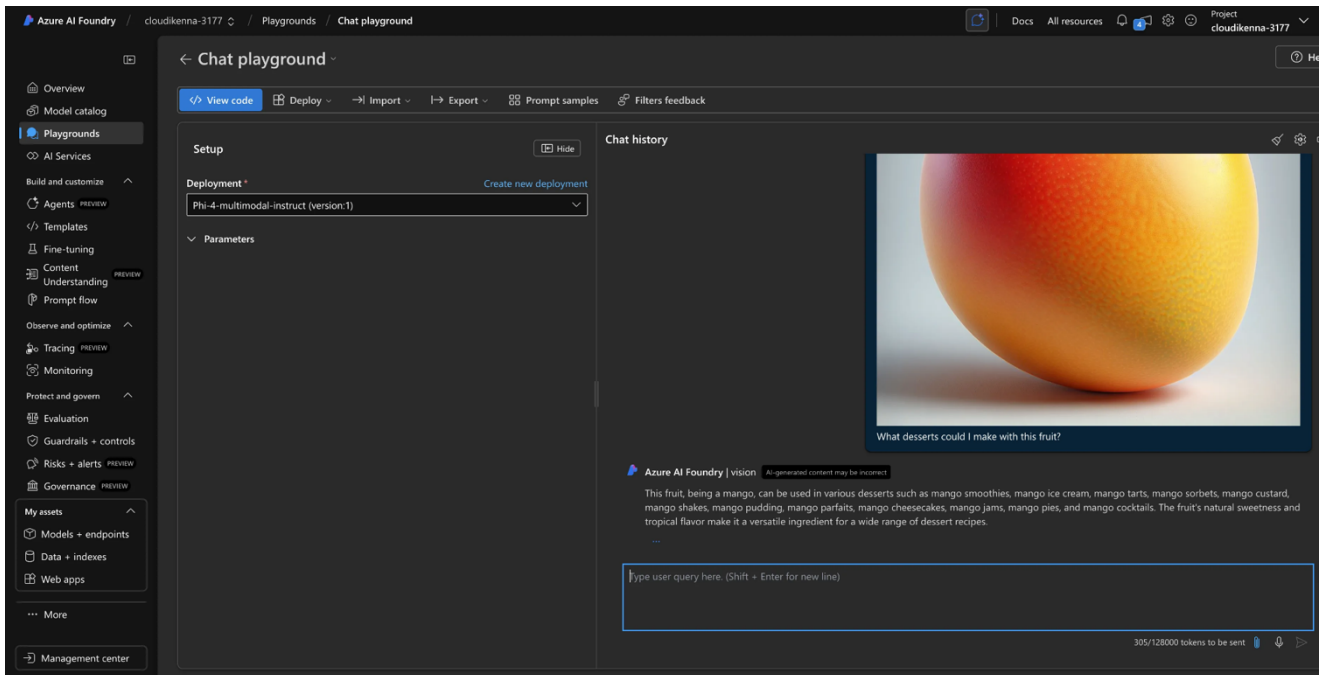
- `cd chat-assistant`
- `pip install -r requirements.txt`

- Configure `audiochat-app.py`

- Selecting and Creating Model



- Testing Model in Playground



This project showcases the complete journey from identifying business problems to deploying enterprise AI solutions—demonstrating both technical depth and practical business acumen that directly translates to value in any organization adopting AI technologies.